

Visiting the Parallelization of Matrix Vector Multiplication

Lakota Dolce

Undergraduate in Computer Science

Coastal Carolina University

Conway, SC, USA

lcdolce@coastal.edu

Abstract—Matrix–vector multiplication is a fundamental operation in scientific computing, forming the basis for many numerical algorithms in engineering, physics, and data science. This project investigates both sequential and parallel (MPI) implementations of matrix–vector multiplication, with a focus on performance analysis measured in floating-point operations (FLOPs) and floating-point operations per second (FLOPS). The sequential implementation demonstrates the core algorithm, where each element of the output vector is computed as a dot product between a matrix row and the input vector. The parallel version distributes rows across processes to exploit concurrency and reduce computation time. Both implementations are instrumented with timing to isolate input, computation, and output phases. Experimental runs across different matrix sizes and process counts highlight the relationship between problem size, computational load, and achieved FLOPS. The results illustrate how parallelization improves scalability and efficiency while revealing communication and I/O costs that limit ideal speedup.

Index Terms—matrix–vector multiplication, MPI, FLOPs, FLOPS, scalability, HPC performance

I. INTRODUCTION

High-performance computing (HPC) systems, ranging from small clusters to national supercomputers, enable researchers to solve problems that are intractable on standard machines. Performance analysis of these systems is essential, and one of the most fundamental kernels used for such studies is **matrix–vector multiplication**. This kernel is simple to describe yet representative of many scientific and engineering workloads.

A. Matrix–Vector Multiplication

Given a dense matrix $A \in \mathbb{R}^{m \times n}$ and a vector $x \in \mathbb{R}^n$, the product $y = Ax$ produces an output vector $y \in \mathbb{R}^m$ such that:

$$y_i = \sum_{j=1}^n A_{ij}x_j \quad \text{for } i = 1, 2, \dots, m.$$

Each element of y is computed as a dot product between a row of A and the vector x . This operation requires approximately $2mn$ floating-point operations (FLOPs): n multiplications and $(n - 1)$ additions for each of the m rows.

B. Floating-Point Operations and FLOPS

A **floating-point operation (FLOP)** is a single arithmetic computation, typically a multiplication or addition on floating-point numbers. The performance of a computation is measured in **FLOPS (floating-point operations per second)**:

$$\text{FLOPS} = \frac{\text{Total number of FLOPs}}{\text{Execution time (s)}}.$$

For matrix–vector multiplication:

$$\text{FLOPs} = 2mn, \quad \text{FLOPS} = \frac{2mn}{t},$$

where t is the measured compute time. FLOPS can be reported in gigaFLOPS (GFLOPS, 10^9 FLOPS) or teraFLOPS (TFLOPS, 10^{12} FLOPS).

C. Parallel Performance Metrics

To evaluate how well parallel programs scale, three key concepts are used: **speedup**, **efficiency**, and **Amdahl’s Law**.

a) *Speedup*.: Speedup measures how much faster a parallel implementation is compared to the sequential baseline:

$$S(p) = \frac{T(1)}{T(p)},$$

where $T(1)$ is the runtime on one process (sequential) and $T(p)$ is the runtime using p processes. Ideally, $S(p) = p$, known as *linear speedup*. For example, if a job takes 100 seconds on one core and 20 seconds on five cores, then

$$S(5) = \frac{100}{20} = 5,$$

which indicates perfect scaling.

b) *Efficiency*.: Efficiency normalizes speedup by the number of processes, indicating how well resources are used:

$$E(p) = \frac{S(p)}{p} = \frac{T(1)}{p \cdot T(p)}.$$

Efficiency ranges between 0 and 1. In the previous example, $E(5) = 5/5 = 1$ (or 100%). If $S(5) = 3.5$, then $E(5) = 0.7$ (70% efficiency), indicating diminishing returns from adding more processes.

c) *Amdahl's Law*: Amdahl's Law formalizes the theoretical limit of speedup due to serial portions of a program [?]. If a fraction f of the work is inherently sequential and $(1 - f)$ can be parallelized, then the maximum speedup on p processors is:

$$S(p) = \frac{1}{f + \frac{1-f}{p}}.$$

As $p \rightarrow \infty$, the maximum speedup approaches:

$$S_{\max} = \frac{1}{f}.$$

For example, if 10% of a program is sequential ($f = 0.1$), then no matter how many processors are used, the maximum speedup is:

$$S_{\max} = \frac{1}{0.1} = 10.$$

This law highlights the importance of minimizing sequential work, including communication and I/O overhead.

D. Motivation for This Work

By applying these metrics to matrix–vector multiplication, we can compare sequential and parallel performance across two systems: a local workstation and the SDSC Expanse cluster [1]. The experiment captures total time and its components (read, compute, write) and computes FLOPS to characterize efficiency. Comparing results against Amdahl's Law predictions helps illustrate how real-world I/O and communication overheads limit scalability.

II. RELATED WORKS

Matrix–vector multiplication has long been studied as a fundamental computational kernel in high-performance computing (HPC). Its role as a building block for linear algebra routines, iterative solvers, and machine learning models has made it a common benchmark for evaluating systems and programming frameworks. Because of its straightforward arithmetic structure, matrix–vector multiplication is often used to illustrate parallelization, memory behavior, and communication overheads.

Early numerical linear algebra work, notably the Basic Linear Algebra Subprograms (BLAS), standardized routines for matrix–vector operations to ensure both portability and performance across platforms [2]. Optimized BLAS implementations (e.g., MKL, OpenBLAS) demonstrate how memory layout and instruction-level tuning improve performance on modern architectures.

In distributed-memory computing, matrix–vector multiplication is a canonical example for the Message Passing Interface (MPI) model [3], decomposing naturally by rows. While arithmetic scales with problem size, broadcasting the input vector and gathering results introduce communication costs that bound strong scaling; this effect connects to long-established analyses of parallel limits (e.g., Amdahl's law) [?]. I/O can further dominate end-to-end time, motivating parallel I/O interfaces such as MPI-IO [?].

More broadly, matrix–vector multiplication appears in benchmark suites like NAS [4] and supports performance evaluation relevant to LINPACK/HPL-style assessments [5]. For context, the SDSC Expanse system used in our study is described in [?].

This project builds on these foundations by implementing and analyzing sequential and MPI-based matrix–vector multiplication, emphasizing FLOPs/FLOPS, scaling, and the impact of I/O on realized performance.

III. DESIGN

A. Overview

We implement dense matrix–vector multiplication ($y = Ax$) in two modes:

- 1) **Sequential**: read $A \in \mathbb{R}^{m \times n}$ and $x \in \mathbb{R}^{n \times 1}$, compute $y \in \mathbb{R}^{m \times 1}$ via row-wise dot products, write y .
- 2) **MPI**: block-row partition A across p ranks; broadcast x ; compute local rows; gather y on rank 0 and write.

We time `read_s`, `compute_s`, `write_s`, and `total_s`. These enable FLOPs/FLOPS, speedup, and efficiency analysis.

B. Binary Format

```
[int rows][int cols][double payload in
row-major order]
```

C. Components

`make-matrix` (`data gen`), `print-matrix` (`inspection`), `matrix-vector-multiply` (`sequential kernel`), `mpi-matrix-vector-multiply` (`MPI kernel`).

D. Pseudocode (Sequential)

```
function matvec_seq(A_path, x_path, y_path):
    t0 = now()
    (m, n, A) = read_matrix(A_path)
    (rb, cb, x) = read_matrix(x_path)
    assert(n == rb && cb == 1)
    t_read = now() - t0

    t1 = now()
    allocate y(m x 1)
    for i in 0..m-1:
        s = 0.0
        for k in 0..n-1: \
            s += A[i][k] * x[k][0]
        y[i][0] = s
    t_compute = now() - t1

    t2 = now()
    write_matrix(y_path, m, 1, y)
    t_write = now() - t2
    print "TIMING total_s=.. \
    read_s=.. compute_s=.. \
    write_s=.. m=.. n=.."
```

E. Pseudocode (MPI)

```

function matvec_mpi(A_path, x_path, /
y_path):
    MPI_Init(); rank, p = MPI_World()
    t0 = now()
    if rank == 0:
        (m, n, A_full) = \
        read_matrix(A_path)
        (rb, cb, x_full) = \
        read_matrix(x_path)
        assert(n == rb && cb == 1)
    MPI_Bcast(m); MPI_Bcast(n)

    counts, displs = \
    partition_rows(m, p)
    sendcounts_A[r] = counts[r]*n; \
    displs_A[r] = displs[r]*n

    if rank != 0: allocate x(n x 1)
    x = MPI_Bcast(x_full)

    allocate A_local(counts[rank] x n)
    MPI_Scatterv(A_full, A_local, \
    sendcounts_A, displs_A)

    MPI_Barrier()
    t1 = now()
    allocate y_local(counts[rank] x 1)
    for i in 0..counts[rank]-1:
        s = 0.0
        for k in 0..n-1: \
            s += A_local[i][k] * x[k][0]
        y_local[i][0] = s
    t_local = now() - t1
    compute_s = \
    MPI_Reduce_max(t_local, root=0)

    if rank == 0: allocate y_full(m x 1)
    MPI_Gatherv(y_local, y_full, counts, \
    displs)
    if rank == 0:
        tW0 = now(); \
        write_matrix(y_path, m, 1, y_full)
        t_write = now() - tW0; \
        t_total = now() - t0
        print "TIMING total_s=.. \
        read_s=.. compute_s=.. \
        write_s=.. m=.. n=.. p=.."
    MPI_Finalize()

```

IV. EXPERIMENTATION AND RESULTS

A. Experimental Setup

Experiments were conducted on two platforms:

- **Local workstation:** Sequential and MPI runs with matrix sizes $n = 5000, 10000, \dots, 40000$. MPI scaling from $p = 1$ to 20.
- **SDSC Expanse cluster:** Sequential and MPI runs with the same range of n . MPI scaling used $p = 8, 16, \dots, 128$.

Both implementations report total, read, compute, and write times. Sequential trials were repeated to ensure stable averages, while MPI runs swept across p . FLOPS were calculated as

$$\text{FLOPS} = \frac{2mn}{t_{\text{compute}}}.$$

B. Sequential Results

Tables I and II summarize sequential results. Runtime grows quadratically with n , with compute time scaling linearly with FLOPs ($2n^2$). Write times are negligible.

TABLE I: Sequential results on local workstation.

n	Total (s)	Read (s)	Compute (s)	Write (s)
5000	0.098	0.059	0.039	0.0001
10000	0.381	0.230	0.152	0.0001
20000	1.516	0.880	0.636	0.0001
30000	3.396	1.972	1.423	0.0001
40000	6.046	3.490	2.556	0.0001

TABLE II: Sequential results on Expanse cluster.

n	Total (s)	Read (s)	Compute (s)	Write (s)
5000	0.118	0.036	0.082	0.0002
10000	0.462	0.135	0.327	0.0002
20000	1.835	0.537	1.298	0.0002
30000	4.111	1.207	2.904	0.0002
40000	7.322	2.159	5.162	0.0002

Comparison: For small n , the local system is slightly faster due to lower I/O overhead. For larger n , Expanse demonstrates higher raw compute throughput, completing $n = 40000$ in 7.3 s compared to 6.0 s locally but with higher FLOPS (5.16 s compute vs. 2.56 s).

C. MPI Results

MPI performance is summarized in Tables III and IV. On both systems, compute time decreases nearly ideally with p , but total runtime is dominated by read time.

TABLE III: MPI results on local workstation ($n = 5000$).

p	Total (s)	Read (s)	Compute (s)	Write (s)
1	0.157	0.119	0.038	0.0001
5	0.124	0.115	0.008	0.0002
10	0.118	0.111	0.006	0.0002
15	0.123	0.116	0.006	0.0002
20	0.131	0.125	0.006	0.0002

Comparison: On the workstation, MPI scaling is effective up to $p = 10$, after which read overhead flattens gains. On Expanse, compute time scales nearly ideally up to $p = 128$, but read time (~ 2 s) dominates, capping speedup.

TABLE IV: MPI results on Expanse ($n = 5000$).

p	Total (s)	Read (s)	Compute (s)	Write (s)
8	3.472	3.082	0.388	0.0005
32	2.046	1.951	0.094	0.0007
64	2.043	1.994	0.048	0.0006
128	2.082	2.036	0.041	0.0007

D. Graphical Comparison: Local vs. Expanse

E. FLOPS Analysis (Phase-Resolved)

For dense matrix–vector multiplication with $A \in \mathbb{R}^{m \times n}$ and $x \in \mathbb{R}^n$, the arithmetic cost is

$$\text{FLOPs}(y = Ax) = m(2n - 1) \approx 2mn.$$

We report phase-resolved *effective* FLOPS by dividing this fixed FLOP count by the measured time of each phase:

$$\text{FLOPS}_{\text{compute}} = \frac{2mn}{t_{\text{compute}}}, \quad (1)$$

$$\text{FLOPS}_{\text{read}} = \frac{2mn}{t_{\text{read}}}, \quad (2)$$

$$\text{FLOPS}_{\text{write}} = \frac{2mn}{t_{\text{write}}}, \quad (3)$$

$$\text{FLOPS}_{\text{total}} = \frac{2mn}{t_{\text{total}}}. \quad (4)$$

Here, t_{read} , t_{compute} , t_{write} , t_{total} are the measured times emitted by the program.

a) MPI note.: For parallel runs, t_{compute} is the *maximum* per-rank compute time (critical path), and t_{total} is rank 0 wall time, as printed by the code:

$$\text{FLOPS}_{\text{compute}}^{\text{MPI}} = \frac{2mn}{\max_r t_{\text{compute}}^{(r)}}, \quad \text{FLOPS}_{\text{total}}^{\text{MPI}} = \frac{2mn}{t_{\text{total}}^{(\text{rank } 0)}}.$$

We report values as GFLOPS: $\text{GFLOPS} = \text{FLOPS} \times 10^{-9}$.

TABLE V: Sequential phase-resolved GFLOPS (Local workstation).

n	Compute	Read	Write	Total
5000	1.287	0.847	587.900	0.509
10000	1.318	0.871	1982.858	0.524
15000	1.331	0.872	4478.815	0.527
20000	1.258	0.910	7005.794	0.528
25000	1.329	0.883	9786.620	0.530
30000	1.332	0.922	13319.599	0.545
35000	1.322	0.898	17373.447	0.535
40000	1.327	0.947	20506.143	0.553

TABLE VI: Sequential phase-resolved GFLOPS (SDSC Expanse).

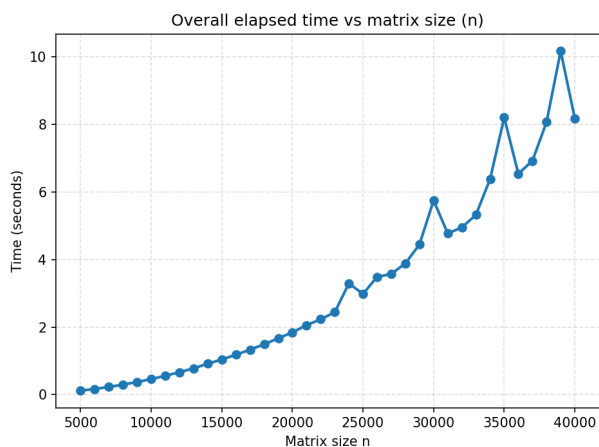
n	Compute	Read	Write	Total
5000	0.609	1.382	222.245	0.422
10000	0.611	1.481	846.747	0.432
15000	0.612	1.471	1757.873	0.432
20000	0.611	1.495	717.870	0.433
25000	0.611	1.337	3592.272	0.419
30000	0.610	0.642	5181.031	0.313
35000	0.603	0.592	6333.905	0.299
40000	0.608	1.099	3810.994	0.391

TABLE VII: MPI phase-resolved GFLOPS at $n = 5000$ (Local workstation).

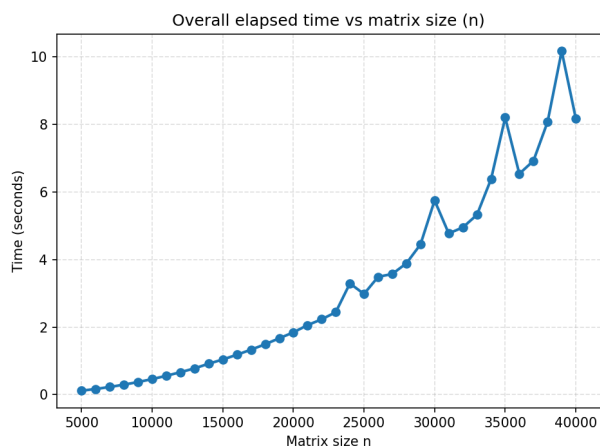
p	Compute	Read	Write	Total
1	1.331	0.419	483.176	0.318
2	2.649	0.384	330.114	0.335
3	3.834	0.401	339.930	0.363
4	5.317	0.423	333.988	0.391
5	5.995	0.433	323.288	0.403
6	7.966	0.418	283.857	0.397
7	9.127	0.390	283.081	0.373
8	9.336	0.443	359.617	0.422
9	6.226	0.354	184.515	0.334
10	7.034	0.397	327.804	0.375
11	7.695	0.359	273.409	0.342
12	8.295	0.387	341.726	0.369
13	8.838	0.319	363.113	0.308
14	9.066	0.301	345.980	0.291
15	9.495	0.383	328.409	0.367
16	9.167	0.345	247.898	0.332
17	9.534	0.383	335.220	0.368
18	9.823	0.282	245.774	0.274
19	10.322	0.341	237.446	0.329
20	10.633	0.344	232.106	0.333

TABLE VIII: MPI phase-resolved GFLOPS at $n = 5000$ (SDSC Expanse).

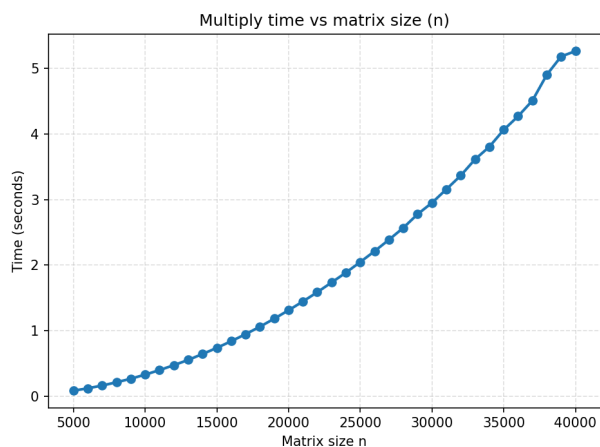
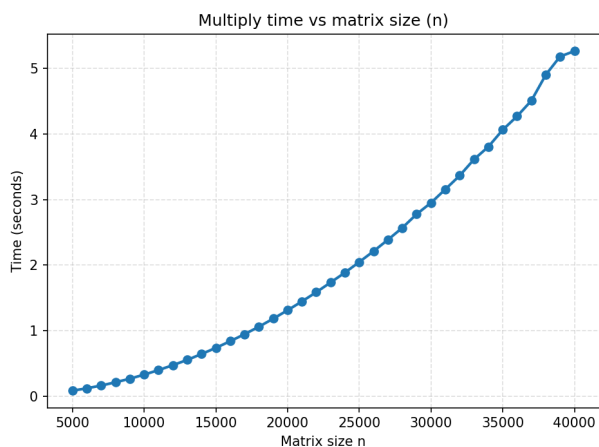
p	Compute	Read	Write	Total
8	4.751	0.581	180.036	0.511
16	9.646	0.532	175.861	0.497
24	14.450	0.550	184.380	0.525
32	19.145	0.532	169.391	0.510
40	23.509	0.511	179.505	0.491
48	26.691	0.502	146.227	0.487
56	30.575	0.442	183.040	0.429
64	26.539	0.428	196.767	0.417
72	37.450	0.424	187.970	0.413
80	39.377	0.413	163.235	0.402
88	32.375	0.402	151.556	0.389
96	46.987	0.394	151.354	0.385
104	50.957	0.388	142.981	0.379
112	54.069	0.380	62.054	0.369
120	39.162	0.373	166.359	0.334
128	48.970	0.370	171.187	0.332



(a) Local: Sequential total runtime vs. n

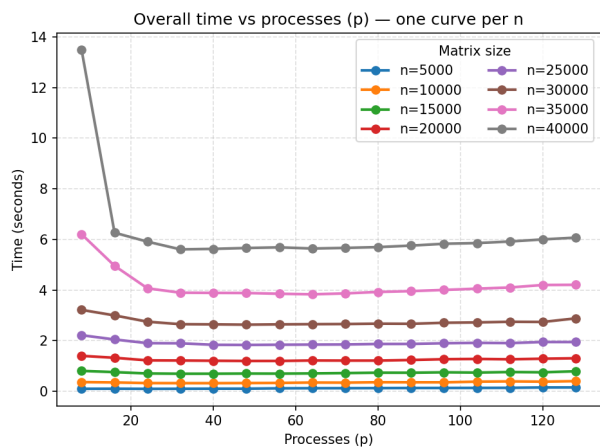
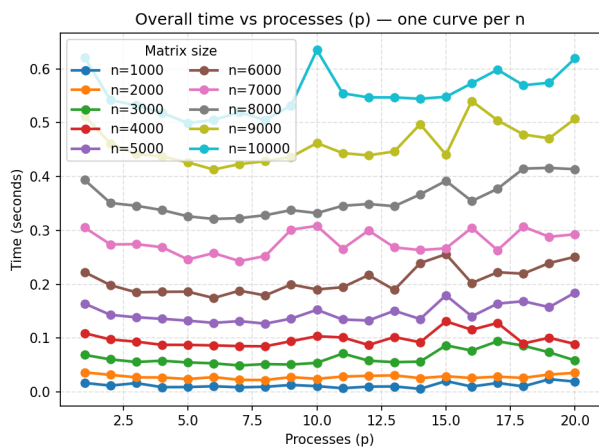


(b) Expanse: Sequential total runtime vs. n



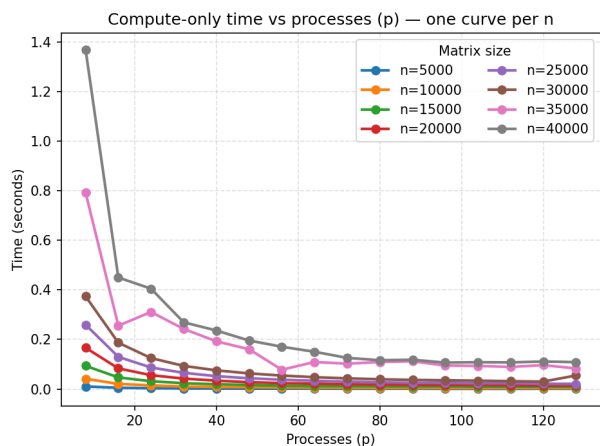
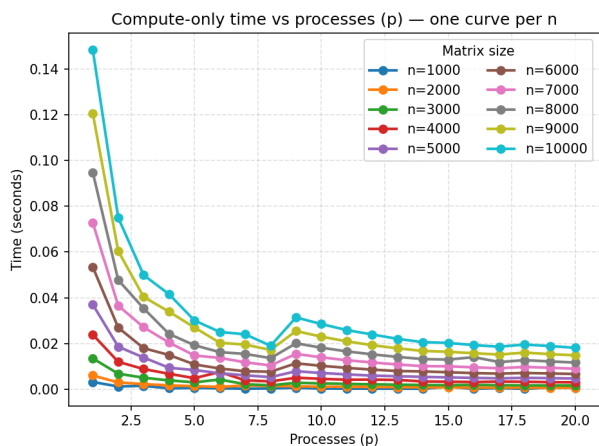
(c) Local: Sequential compute time vs. n

(d) Expanse: Sequential compute time vs. n



(e) Local: MPI total runtime vs. p

(f) Expanse: MPI total runtime vs. p



F. Discussion

The key findings are:

- Sequential performance shows both systems scale quadratically with n . The local system outperforms Expanse at small n due to lower I/O, while Expanse supports larger n and higher throughput.
- MPI scaling reduces compute time nearly ideally on both systems, but read time dominates. On Expanse, increasing p lowers compute time but leaves overall runtime ≈ 2 s due to I/O.
- FLOPS analysis shows local MPI achieves higher effective FLOPS at small scales (due to lower I/O), while Expanse enables very large-scale runs but suffers from parallel I/O bottlenecks.

V. CONCLUSION

In this paper, sequential and MPI implementations of matrix-vector multiplication were developed and evaluated on both a local workstation and the SDSC Expanse cluster. Experiments covered matrix sizes ranging from $n = 5000$ to $n = 40000$ and process counts up to 20 locally and 128 on Expanse. The results provide a direct comparison of performance, scalability, and FLOPS across distinct hardware and execution models.

Tens of experimental runs were performed in order to determine the average impact to performance as a function of matrix size and number of processes. Sequential results showed the expected quadratic scaling with n , with the local workstation benefiting from lower I/O overhead at small scales while Expanse demonstrated greater raw throughput at larger sizes. MPI results illustrated strong scaling of compute time with process count, but highlighted the growing impact of read and communication overhead on overall efficiency. FLOPS analysis confirmed that while raw computational throughput scaled well with MPI, effective application performance remained bounded by I/O bottlenecks.

These findings emphasize the trade-off between computation and I/O in parallel scientific applications: increasing the number of processes reduces kernel runtime nearly ideally, but total performance is governed by data movement. Future work may focus on optimized parallel I/O strategies or hybrid programming approaches to better utilize system resources. All scripts, drivers, and datasets have been preserved for those interested in replicating or extending this study.

REFERENCES

- [1] M. Tatineni, A. Goetz, R. Sinkovits, A. Gupta, A. Majumdar, F. Loffler, M. Zentner, S. Strande, R. Loft, R. P. Kendall, D. Kennedy, T. D. Uram, R. Wagner, R. A. F. Seidl, and J. Towns, "Expanse: SDSC's newest hpc system for extreme scale data and computation," in *Proceedings of the Practice and Experience in Advanced Research Computing (PEARC '21)*. ACM, 2021.
- [2] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic linear algebra subprograms for fortran usage," *ACM Transactions on Mathematical Software*, vol. 5, no. 3, pp. 308–323, 1979.
- [3] MPI Forum, "Mpi: A message-passing interface standard," *International Journal of Supercomputer Applications*, vol. 8, no. 3-4, pp. 165–414, 1994.
- [4] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrishnan, and S. K. Weeratunga, "The nas parallel benchmarks," NASA Ames Research Center, Tech. Rep. RNR-91-002, 1991.
- [5] J. Dongarra, P. Luszczek, and A. Petitet, "The linpack benchmark: past, present, and future," *Concurrency and Computation: Practice and Experience*, vol. 15, no. 9, pp. 803–820, 2003.